LAB 04

Thiết kế bộ tăng tốc Conv1 INT8

Vitis HLS và Vivado trên SoC PYNQ-Z2

Slide 05 · Kiến thức nền, quy trình triển khai và kết quả

Nội dung bài học

- Lựa chọn Conv1 + ReLU + MaxPool để tăng tốc trên FPGA.
- Thiết kế giao tiếp luồng AXI và IP bằng Vitis HLS.
- Kiểm chứng C Simulation, C Synthesis và Co-Simulation.
- Tích hợp Zynq PS – AXI DMA – IP trong Vivado.
- Kiểm tra Timing, tài nguyên và tạo bitstream.

Tại sao cần tăng tốc Conv1 trên FPGA?

- Mạng Tiny CNN V5 được huấn luyện và lượng tử hóa INT8 từ Lab 03.
- Conv1 áp dụng 8 bộ lọc 3 × 3 trên ảnh MRI xám 96 × 96.
- Conv1 có nhiều phép nhân–cộng lặp lại, phù hợp xử lý song song.
- Lab 04 chỉ làm phần đầu của CNN bằng FPGA; các lớp sau dành cho ARM.

Ở Lab 04, chưa có số đo tốc độ chạy thực tế trên bo mạch PYNQ-Z2.

Ranh giới phân chia ARM và FPGA

FPGA xử lý đến hết MaxPool1, sau đó đưa tensor INT8 cho ARM.

Tầng	Kích thước đầu ra	Nơi xử lý
Ảnh đầu vào	96 × 96 × 1	ARM chuẩn bị
Conv1 + ReLU	94 × 94 × 8	FPGA
MaxPool1	47 × 47 × 8	FPGA
Các lớp CNN còn lại	4 điểm số phân loại	ARM (Lab 05)

Lượng dữ liệu qua bộ tăng tốc

9.216

byte INT8 đầu vào

17.672

byte INT8 đầu ra

- Đầu vào gồm **96 × 96** pixel; mỗi pixel là một số nguyên 8-bit.
- Sau MaxPool: **47 × 47 × 8 = 17.672** phần tử INT8.

Dữ liệu đầu ra đóng gói 8 kênh × 8 bit = 64 bit cho mỗi vị trí pooling.

Lượng tử hóa INT8 là gì?

- **INT8**: mỗi phần tử dữ liệu được lưu bằng số nguyên có dấu 8-bit.
- **Scale** và **zero-point**: tham số chuyển giữa giá trị thực và giá trị lượng tử.
- **Bias**: dùng INT32; trọng số Conv1 là INT8 với hệ số lượng tử hóa theo từng kênh.
- **Requantization**: đổi kết quả tích lũy INT32 về INT8 đầu ra.

$$\text{Giá trị thực} \approx (\text{giá trị INT8} - \text{zero-point}) \times \text{scale}$$

Trích tham số Conv1 từ mô hình INT8

- Nguồn: **tiny_cnn_v5_int8.tflite**, xử lý bằng script **extract_conv1.py**.
- Trọng số theo TFLite: **[8, 3, 3, 1]**; khi vào HLS dùng **[8][3][3]**.
- Giữ đúng weight, bias, multiplier và shift theo từng kênh đầu ra.

Tham số lượng tử hóa	Giá trị trong Lab 04
Input zero-point / scale	−128 / xấp xỉ 0,0039215689
Output zero-point / scale	−128 / xấp xỉ 0,0039095692

Tệp tham số conv1_int8_params.h

```
#define CONV1_IN_ZERO_POINT  (-128)

#define CONV1_OUT_ZERO_POINT (-128)

#define CONV1_FILTERS        8

#define CONV1_KERNEL         3

static const int8_t conv1_w_int8[8][3][3] = { /* ... */ };

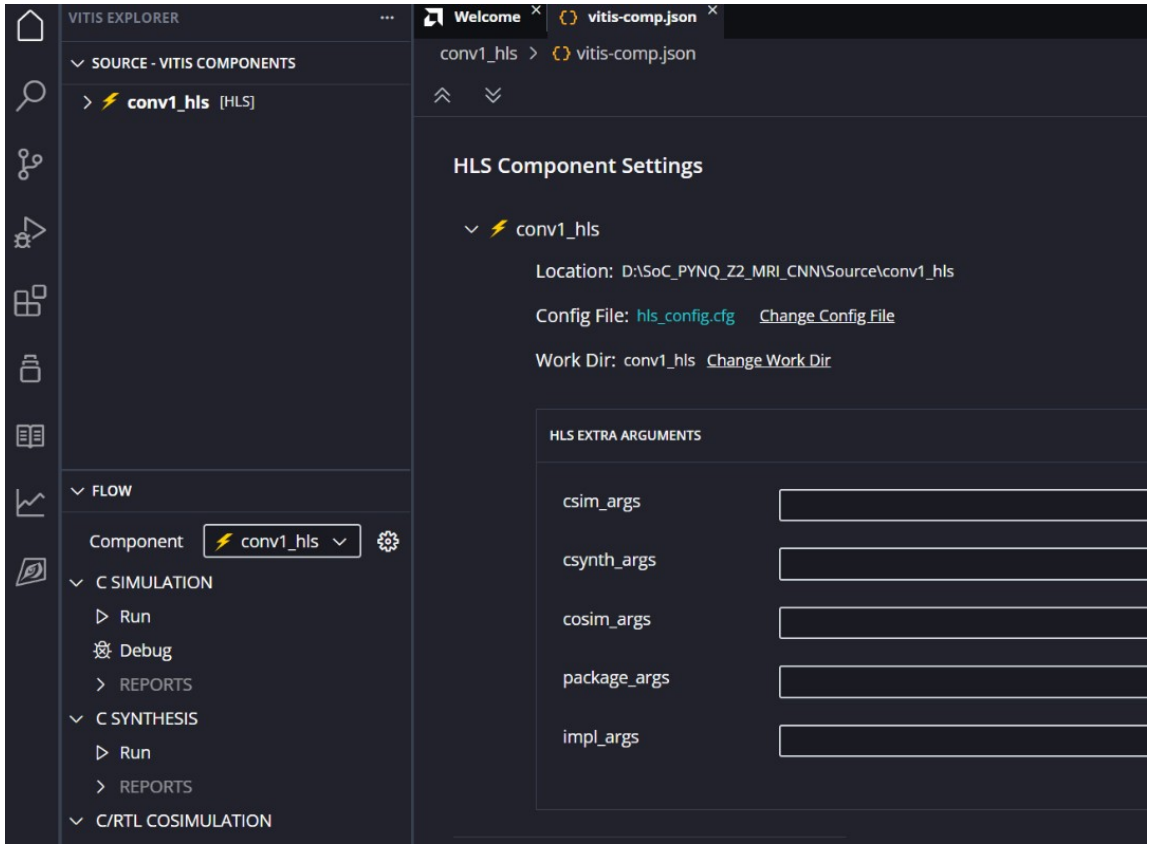
static const int32_t conv1_b_int32[8] = { /* ... */ };
```

Đây là trích đoạn khai báo trong báo cáo, không phải toàn bộ mảng trọng số. Cấu hình Q1.15 cũ không dùng cho bitstream cuối.

Vitis HLS chuyển C/C++ thành phần cứng

- **HLS (High-Level Synthesis)**: tổng hợp mô tả C/C++ thành mạch RTL.
- **IP (Intellectual Property)**: khối phần cứng đóng gói để đưa vào Vivado.
- Dự án tạo HLS Component **conv1_hls** trong Vitis Unified IDE 2026.1.
- Mục tiêu: thiết bị **xc7z020clg400-1**, clock 10 ns (100 MHz).

Tạo HLS Component conv1_hls



Ảnh giao diện Vitis Unified IDE trong Lab 04: HLS Component conv1_hls.

Ba tệp quan trọng trong Vitis HLS

Tệp	Vai trò
conv1.cpp	Cài đặt thuật toán và các cổng giao tiếp IP.
conv1_int8_params.h	Trọng số, bias và tham số lượng tử hóa INT8.
conv1_tb.cpp	Testbench kiểm chứng đầu ra bằng CPU reference.

Top-level function: **conv1_hls** · Package format: **ip_catalog**.

AXI4-Stream và AXI4-Lite khác nhau thế nào?

- **AXI4-Stream**: truyền dữ liệu lần lượt qua cổng s_axis và m_axis.
- **AXI4-Lite**: ARM ghi/đọc thanh ghi để điều khiển IP.
- Các tín hiệu điều khiển liên quan: **ap_start, ap_done, ap_idle**.

Giao tiếp	Độ rộng	Chức năng
s_axis	8 bit	Nhận từng pixel INT8
m_axis	64 bit	Xuất 8 kênh INT8 đã pooling

Khai báo giao tiếp luồng trong HLS

```
typedef ap_axiu<8, 0, 0, 0> in_axis_t;

typedef ap_axiu<64, 0, 0, 0> out_axis_t;


void conv1_hls(hls::stream<in_axis_t>& s_axis,

               hls::stream<out_axis_t>& m_axis) {

#pragma HLS INTERFACE mode=axis port=s_axis

#pragma HLS INTERFACE mode=axis port=m_axis

#pragma HLS INTERFACE mode=s_axilite port=return bundle=control

    // Streaming computation

}
```

Trích đoạn giao tiếp từ Lab 04. Luồng vào 8-bit, luồng ra 64-bit, điều khiển AXI4-Lite.

Kết quả báo cáo giao tiếp HLS

INTERFACE	REGISTER	OFFSET	WIDTH	ACCESS	DESCRIPTION
s_axi_control	CTRL	0x00	32	RW	Control signals
s_axi_control	GIER	0x04	32	RW	Global Interrupt Enable Register
s_axi_control	IP_IER	0x08	32	RW	IP Interrupt Enable Register
s_axi_control	IP_ISR	0x0c	32	RW	IP Interrupt Status Register

AXIS

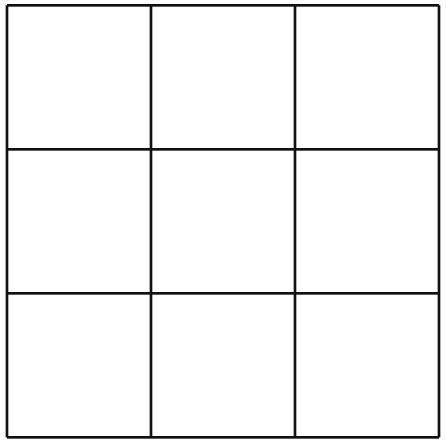
INTERFACE	DIRECTION	REGISTER MODE	TDATA	TKEEP	TLAST	TREADY
m_axis	out	both	64	8	1	1
s_axis	in	both	8	1	1	1

TOP LEVEL CONTROL

Bảng interface Vitis HLS: cổng m_axis có TDATA 64-bit; s_axis có TDATA 8-bit.

Line buffer và cửa sổ trượt 3 × 3

Không cần giữ toàn bộ tensor Conv1 trung gian trong BRAM.



Cửa sổ 3 × 3

8 bộ lọc
tính tích chập song song

– Lưu hai hàng ảnh trước bằng **line buffer** và cập nhật cửa sổ mỗi khi nhận pixel mới.

Minh họa nguyên lý, không phải ảnh chụp thiết kế HLS của nhóm.

Pipeline và Unroll trong HLS

- **PIPELINE**: cho các lần lặp của vòng xử lý gối đầu nhau.
- **II (Initiation Interval)**: khoảng chu kỳ giữa hai lần bắt đầu xử lý.
- **UNROLL**: nhân bản logic vòng lặp để tăng mức song song.
- Trong Lab 04, mục tiêu vòng đọc ảnh: **II = 1**.

```
#pragma HLS PIPELINE II=1
// One new pixel can start each clock
```

Requantization và hợp nhất ReLU

```
#pragma HLS PIPELINE II=1
ap_int<32> acc = conv1_b_int32[f];
acc += (ap_int<32>)prod;
ap_int<32> q = requantize(
    acc, conv1_multiplier[f], conv1_shift[f]);
q += CONV1_OUT_ZERO_POINT;
if (q < -128) q = -128;
if (q > 127) q = 127;
```

Theo Lab 04, zero-point đầu ra = −128 là mã biểu diễn giá trị thực 0; clamp INT8 đồng thời hợp nhất ReLU.

MaxPooling 2 × 2 và tín hiệu TLAST

- **MaxPooling** lấy giá trị cực đại trong vùng 2 × 2, cho từng kênh.
- Thiết kế chỉ giữ các giá trị cần thiết, không lưu toàn bộ tensor 94 × 94 × 8.
- Đầu ra: 47 × 47 = **2.209 từ 64-bit**, tương đương 17.672 byte.
- **TLAST** chỉ kích hoạt ở từ dữ liệu cuối cùng của một ảnh.

Độ rộng 64-bit = 8 kênh INT8 × 8 bit/kênh. Phần mềm phải giải mã đúng thứ tự byte.

C Simulation kiểm chứng điều gì?

- Testbench tạo đầu vào INT8 có giá trị xác định.
- CPU reference tự tính Conv1 + ReLU + MaxPool và đối chiếu từng kênh.
- Ngoài dữ liệu, testbench còn kiểm tra tín hiệu TLAST.

```
Verified 2209 pooled vectors, 17672 INT8 channels; errors=0
PASS: Conv1 + ReLU + MaxPool INT8 CPU vs HLS C simulation
CSim done with 0 errors
```

PASS nghĩa là CPU reference và HLS C khớp trong testbench; chưa chứng minh khớp TFLite trên ảnh thật.

C Synthesis – đọc báo cáo ước lượng

Estimated Quality of Results

Timing Estimate

TARGET	WITH UNCERTAINTY	ESTIMATED
10.00 ns	7.30 ns	6.955 ns

Performance & Resource Estimates

Modules

DULES & LOOPS	LATENCY(NS)	ITERATION LATENCY	INTERVAL	TRIP COUNT	PIPELINED	STRUCTURE	BRAM	DSP
conv1_hls (1)	9.230E4		9,226		loop auto-rewind s	function	6	58
ROW_LOOP_COL_LOOP	9.228E4	14	1	9,216	yes	loop		

- Clock target: **10 ns**; estimated period: **6,955 ns**.
- II = 1 ở vòng đọc 9.216 pixel; latency toàn IP khoảng **9.230 chu kỳ**.

HLS: thời gian và tài nguyên ước lượng

Đại lượng	Giá trị từ HLS
Target / Estimated period	10 ns / 6,955 ns
Initiation Interval	II = 1
Latency toàn chức năng	Khoảng 9.230 chu kỳ
Tài nguyên IP ước lượng	LUT 5.493; FF 4.856; BRAM 6; DSP 58

9.230 chu kỳ × 10 ns ≈ **92,3 µs / ảnh**

92,3 µs chưa phải thời gian chạy thật

- HLS là **ước lượng thiết kế** trước khi chạy trên PYNQ-Z2.
- Con số 92,3 µs không bao gồm chi phí **AXI DMA**, điều khiển ARM và phần mềm.
- Thời gian end-to-end của hệ thống ARM + FPGA cần đo ở Lab 05/06.

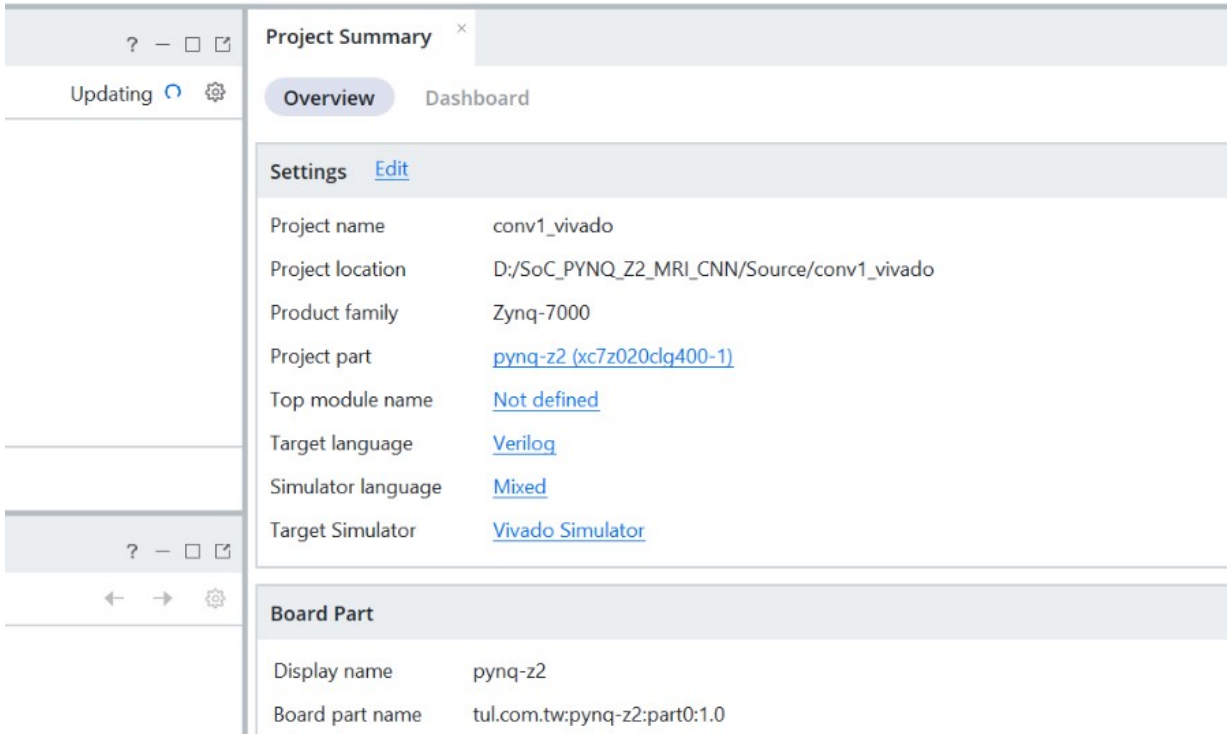
Không so sánh trực tiếp **92,3 µs** với thời gian inference đầy đủ của CNN.

Co-Simulation và đóng gói IP

- **C/RTL Co-Simulation**: so sánh hành vi C/C++ với RTL được tạo từ HLS.
- Báo cáo mô phỏng ghi nhận latency khoảng **9.228 chu kỳ**.
- Package IP theo định dạng **ip_catalog** để Vivado có thể nhận.
- Tệp đầu ra gồm **component.xml** và gói **xilinx_com_hls_conv1_hls_1_0.zip**.

9.228 chu kỳ vẫn là kết quả mô phỏng RTL; chưa phải đo thực tế trên bo mạch.

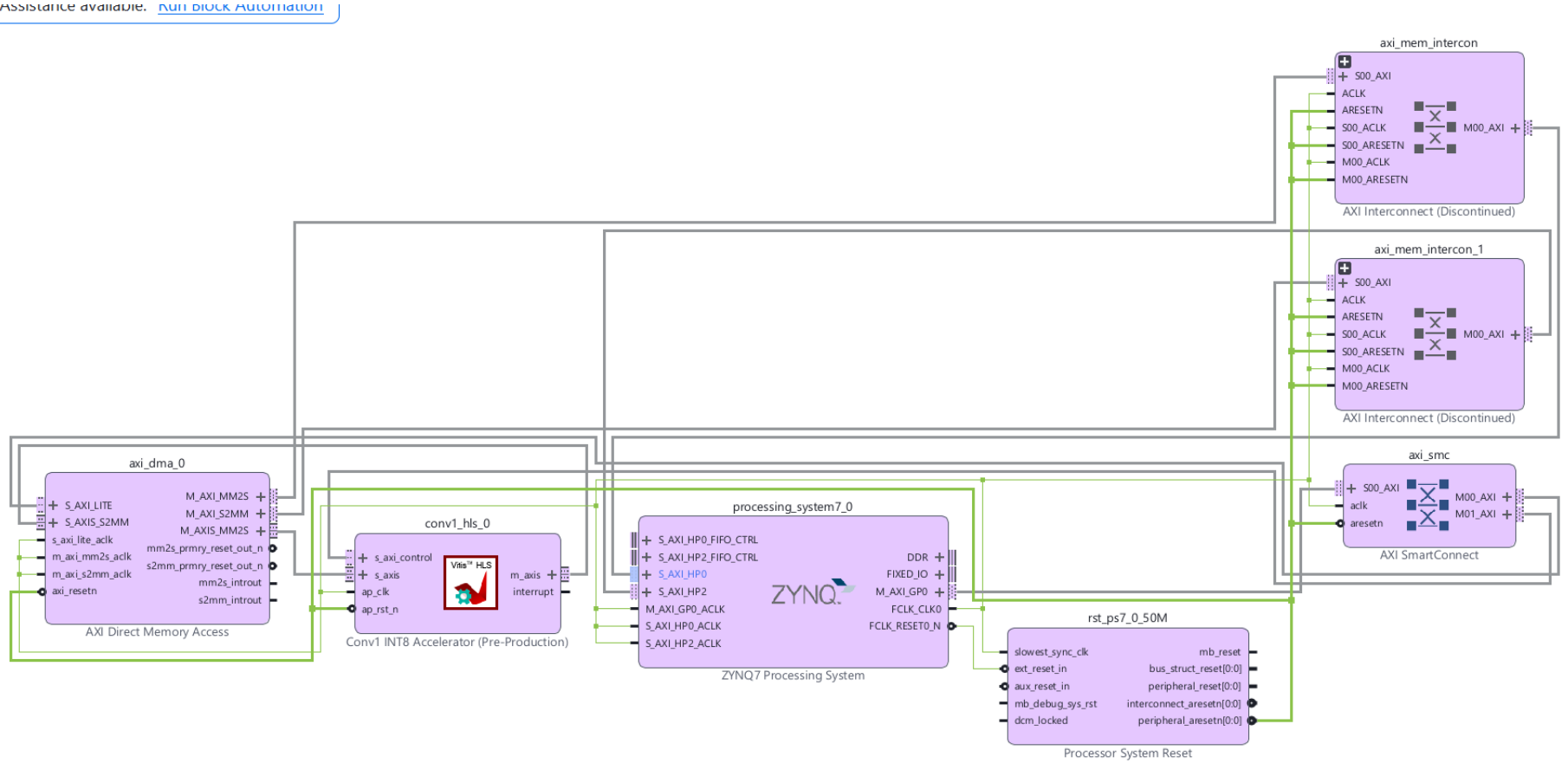
Tạo dự án Vivado và đăng ký IP



- Vivado 2026.1; project **conv1_vivado**; board **pynq-z2**.
- Đăng ký HLS **impl/ip** trong IP Repository.

Block Design kết hợp PS – DMA – IP

Thiết kế conv1_system trên Vivado (hình minh chứng gốc của Lab 04).



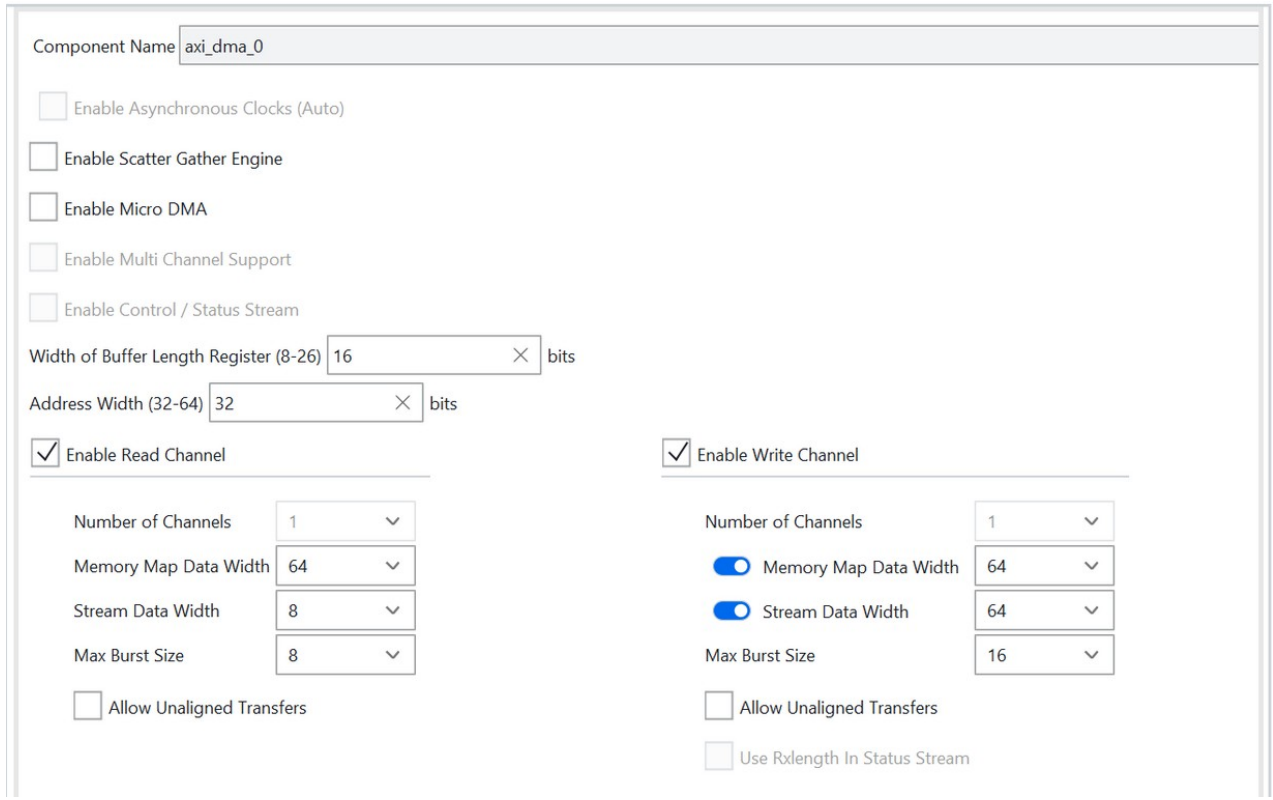
Các khối chính: processing_system7_0, axi_dma_0, conv1_hls_0 và AXI Interconnect/Reset.

Đường truyền dữ liệu qua AXI DMA

- **MM2S** (Memory Mapped to Stream): đọc dữ liệu trong RAM và đẩy đến s_axis của Conv1.
- **S2MM** (Stream to Memory Mapped): nhận kết quả m_axis và ghi lại RAM.
- ARM điều khiển DMA và IP qua **AXI4-Lite** (PS/M_AXI_GP0).

Đường dữ liệu	Kết nối trong thiết kế
MM2S	PS/S_AXI_HP0 → DMA → Conv1/s_axis
S2MM	Conv1/m_axis → DMA → PS/S_AXI_HP2

Cấu hình AXI DMA tương thích với INT8



Simple DMA (tắt Scatter Gather); MM2S Stream 8-bit, S2MM Stream 64-bit; Buffer Length Register 16-bit.

Tại sao chọn Buffer Length 16-bit?

17.672

byte cần ghi vào RAM

16.383

byte tối đa của 14-bit

- Buffer đầu ra **lớn hơn** giới hạn của thanh ghi độ dài 14-bit.
- Lab 04 dùng **Width of Buffer Length Register = 16 bit**, đủ để nhận dữ liệu của một ảnh.

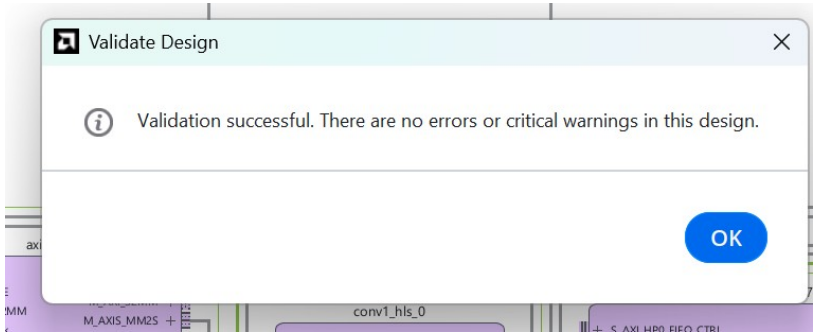
Cấu hình DDR đúng với board PYNQ-Z2

- Giá trị PS ban đầu đọc được: **DDR Bus Width = 32 bit**, chưa phù hợp.
- Trước khi sửa, lưu Block Design bằng **write_bd_tcl** để sao lưu.
- Dùng preset của đúng Board Part để thiết lập DDR/MIO đồng bộ.

Thông số sau preset	Giá trị đã kiểm tra
DDR Bus Width	16 bit
DDR Part Number	MT41J256M16 RE-125
PS Input Clock	50 MHz
FCLK_CLK0	100 MHz

Clock, Reset và Validate Design

- Từ PS, **FCLK_CLK0 = 100 MHz** cấp clock cho DMA, Conv1 IP và các cổng AXI liên quan.
- FCLK_RESET0_N nối với bộ điều khiển reset để đồng bộ reset ngoại vi.
- Chạy **validate_bd_design** sau khi kiểm tra kết nối.



Vivado báo Validate successful; bước này chỉ kiểm tra tính hợp lệ của Block Design.

Synthesis và tài nguyên phần cứng

- Tạo **HDL Wrapper** và Generate Output Products trước khi chạy Synthesis.
- Bảng Utilization dưới đây là của **thiết kế tổng thể**, không chỉ riêng IP.

Tài nguyên	Đã dùng / Tổng có	Tỷ lệ
Slice LUT	4.405 / 53.200	8,28%
Slice Register	4.484 / 106.400	4,21%
Block RAM Tile	6 / 140	4,29%
DSP	62 / 220	28,18%
BUFGCTRL	1 / 32	3,13%

Bảng Utilization từ Vivado Synthesis

Name	Slice LUTs (53200)	Slice Registers (106400)	F7 Muxes (26600)	Block RAM Tile (140)	DSPs (220)	Bonded IOPADs (130)	BUFGCTRL (32)
N conv1_system_wrapper	4405	4484	13	6	62	130	1
I conv1_system_i (conv1_system)	4405	4484	13	6	62	0	1
I axi_dma_0 (conv1_system_axi_	1650	2155	13	3	0	0	0
I axi_mem_intercon (conv1_syste	46	85	0	0	0	0	0
I axi_mem_intercon_1 (conv1_sy	196	203	0	0	0	0	0
I axi_smc (conv1_system_axi_sm	404	441	0	0	0	0	0
I conv1_hls_0 (conv1_system_co	2070	1567	0	3	62	0	0

Ảnh chụp kết quả Synthesized Design trong Lab 04; riêng conv1_hls_0 chiếm 2.070 LUT, 1.567 FF, 3 BRAM và 62 DSP.

Hiểu chỉ số Timing WNS và TNS

- **WNS (Worst Negative Slack)**: độ dư thời gian xấu nhất so với ràng buộc Setup.
- **TNS (Total Negative Slack)**: tổng phần vi phạm Setup Timing.
- Để đánh giá cuối cùng, cần xem Timing **sau Implementation**, vì khi đó đã đặt và đi dây.

Setup WNS sau Implementation = **+1,355 ns**, TNS = **0,000 ns**.

So sánh Timing qua hai giai đoạn

Chỉ số	Sau Synthesis	Sau Implementation
WNS (Setup)	+2,234 ns	+1,355 ns
TNS	0,000 ns	0,000 ns
WHS (Hold)	+0,045 ns	+0,047 ns
THS	0,000 ns	0,000 ns
WPWS	+3,750 ns	+3,750 ns
Failing endpoints	0	0

Implementation WNS dương và không có failing endpoints □ đạt các ràng buộc timing được khai báo.

Timing sau Implementation

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS)	1.355 ns	Worst Hold Slack (WHS)	0.047 ns	Worst Pulse Width Slack (WPWS)	3.750 ns
Total Negative Slack (TNS)	0.000 ns	Total Hold Slack (THS)	0.000 ns	Total Pulse Width Negative Slack (TPWS)	0.000 ns
Number of Failing Endpoints	0	Number of Failing Endpoints	0	Number of Failing Endpoints	0
Total Number of Endpoints	14386	Total Number of Endpoints	14386	Total Number of Endpoints	4786

- WNS Setup **+1,355 ns**; TNS **0,000 ns**.
- Không có failing endpoints theo báo cáo sau Placement & Routing.

DRC và Generate Bitstream

- Chạy **report_drc** trong Implemented Design và kiểm tra kết quả.
- Giá trị trả về Tcl **0** không đủ để tự kết luận tổng số DRC violations = 0.
- Chạy **Generate Bitstream** rồi kiểm tra tiến độ của run impl_1.

```
puts "STATUS: [get_property STATUS [get_runs impl_1]]"
puts "PROGRESS: [get_property PROGRESS [get_runs impl_1]]"
# STATUS: write_bitstream Complete!
# PROGRESS: 100%
```

Bitstream và Hardware Handoff để làm gì?

- **.bit** là cấu hình phần cứng dùng để nạp vào PL của FPGA.
- **.hwh** chứa mô tả kết nối/phần cứng, giúp PYNQ làm việc với Overlay.
- Hai tệp cần giữ cùng tên gốc để PYNQ nhận đúng cặp Overlay.

Tệp kết quả	Kích thước ghi nhận
conv1_accel.bit	4.045.775 byte
conv1_accel.hwh	331.944 byte

Sao chép tệp từ Vivado sang Source

Các bản gốc vẫn nằm trong project Vivado; bản sao phục vụ PYNQ.

```
set src "D:/SoC_PYNQ_Z2_MRI_CNN/Source/conv1_vivado"

set dst "D:/SoC_PYNQ_Z2_MRI_CNN/Source"

set impl "$src/conv1_vivado.runs/impl_1"

set hw "$src/conv1_vivado.gen/sources_1/bd/conv1_system/hw_handoff"

file copy -force "$impl/conv1_system_wrapper.bit" "$dst/conv1_accel.bit"

file copy -force "$hw/conv1_system.hwh" "$dst/conv1_accel.hwh"
```

Lệnh Tcl đã viết lại với biến đường dẫn cho dễ đọc; đích tệp và tác dụng như trong Lab 04.

Lab 04 đã chứng minh được điều gì?

- C Simulation khớp CPU reference ở **2.209 vector, 17.672 phần tử INT8**, errors = 0.
- HLS ước lượng latency khoảng **9.230 chu kỳ** (~92,3 µs tại 100 MHz).
- Implementation thỏa Timing với **WNS +1,355 ns**.
- Bitstream tạo thành công: **write_bitstream Complete!, 100%**.

Điều gì cần kiểm chứng ở Lab 05?

- Nạp cặp **conv1_accel.bit** và **conv1_accel.hwh** trên PYNQ-Z2.
- Truyền dữ liệu thật qua **AXI DMA** và xác minh kết quả đầu ra Conv1.
- Đối chiếu với TFLite Runtime, sau đó ghép với các lớp còn lại trên ARM.
- Đo thời gian thực tế trước khi kết luận mức tăng tốc.

HLS/RTL Simulation và Timing chưa chứng minh hệ thống chạy đúng trên board.

Câu hỏi ôn tập sau Lab 04

- Vì sao Conv1 của Tiny CNN V5 được chọn tăng tốc thay vì đưa toàn mạng lên FPGA?
- AXI4-Stream khác AXI4-Lite ra sao? MM2S và S2MM có nhiệm vụ gì?
- II = 1 và HLS latency khoảng 92,3 µs nói lên điều gì – và không nói lên điều gì?
- Vì sao cần cả bitstream .bit lẫn phần cứng handoff .hwh?